

Assignment number: 1

Title: Table creation (DDL) and Data Insertion

Problem statement:

Create the following tables using the schema given below and insert given data set accordingly.

Table Name- client _master

Description- Used to store client information

Column number	Column name	Data type	Size	Attributes
1	Client_no	Varchar2	6	Primary key, first letter must start with 'C'
2	Name	Varchar2	30	Not NULL
3	Address1	Varchar2	30	
4	Address2	Varchar2	30	
5	City	Varchar2	15	
6	State	Varchar2	15	
7	Pincode	Number	6	
8	Balance_due	Number	10,2	

Data of client_master table

Col-1	Col-2	Col-3	Col-4	Col-5	Col-6	Col-7	Col-8
C001	Ivan Bayross	P-76	Worli	Bombay	Maharastra	400054	15000
C002	VandanaSatiwal	128	Adams Street	Madras	TamilNadu	780001	0
C003	PramadaJaguste	157	Gopalpur	Kolkata	West Bengal	700058	5000
C004	BasuNavindgi	A/12	Nariman	Bombay	Maharastra	400056	0
C005	Ravi Sreedharan	B/34	Rajnagar	Delhi	Delhi	100001	2000
C006	Rukmini	Q-12	Bandra	Bombay	Maharastra	400050	0

Source code and Output:

```
Oracle SQL*Plus
File Edit Search Options Help
SQL> CREATE TABLE client_master (
  2   client_no VARCHAR2(6) PRIMARY KEY,
  3   name VARCHAR2(30) NOT NULL,
  4   address1 VARCHAR2(30),
  5   address2 VARCHAR2(30),
  6   city VARCHAR2(15),
  7   state VARCHAR2(15),
  8   pincode NUMBER(6),
  9   balance_due NUMBER(10,2)
 10 );

Table created.

SQL> desc client_master
Name                                Null?    Type
-----
CLIENT_NO                          NOT NULL VARCHAR2(6)
NAME                               NOT NULL VARCHAR2(30)
ADDRESS1                           VARCHAR2(30)
ADDRESS2                           VARCHAR2(30)
CITY                               VARCHAR2(15)
STATE                             VARCHAR2(15)
PINCODE                           NUMBER(6)
BALANCE_DUE                       NUMBER(10,2)

SQL> INSERT INTO client_master
  2 (client_no, name, address1, address2, city, state, pincode, balance_due)
  3 VALUES
  4 ('C001', 'Ivan Bayross', 'P-76', 'Worli', 'Bombay', 'Maharastra', 400054, 15000);

1 row created.

SQL> INSERT INTO client_master
  2 (client_no, name, address1, address2, city, state, pincode, balance_due)
  3 VALUES
  4 ('C002', 'VandanaSatiwal', '128', 'Adams Street', 'Madras', 'TamilNadu', 780001, 0);

1 row created.

SQL> INSERT INTO client_master
  2 (client_no, name, address1, address2, city, state, pincode, balance_due)
  3 VALUES
  4 ('C003', 'PranadaJaguste', '157', 'Gopalpur', 'Kolkata', 'West Bengal', 700058, 5000);

1 row created.

SQL> INSERT INTO client_master
  2 (client_no, name, address1, address2, city, state, pincode, balance_due)
  3 VALUES
  4 ('C004', 'BasuNavindgi', 'A/12', 'Marinan', 'Bombay', 'Maharastra', 400056, 0);

1 row created.

SQL> INSERT INTO client_master
  2 (client_no, name, address1, address2, city, state, pincode, balance_due)
  3 VALUES
  4 ('C005', 'Ravi Sreedharan', 'B/34', 'Rajnagar', 'Delhi', 'Delhi', 100001, 2000);

1 row created.

SQL> INSERT INTO client_master
  2 (client_no, name, address1, address2, city, state, pincode, balance_due)
  3 VALUES
  4 ('C006', 'Rukmini', 'Q-12', 'Bandra', 'Bombay', 'Maharastra', 400050, 0);

1 row created.

SQL> select * from client_master;

CLIENT NAME                                ADDRESS1                                ADDRESS2                                CITY                                STATE                                PINCODE BALANCE_DUE
-----
C001 Ivan Bayross                          P-76                                  Worli                                  Bombay                                Maharastra                                400054            15000
C002 VandanaSatiwal                      128                                  Adams Street                          Madras                                TamilNadu                                780001              0
C003 PranadaJaguste                      157                                  Gopalpur                             Kolkata                                West Bengal                                700058              5000
C004 BasuNavindgi                        A/12                                  Marinan                              Bombay                                Maharastra                                400056              0
C005 Ravi Sreedharan                      B/34                                  Rajnagar                             Delhi                                  Delhi                                  100001              2000
C006 Rukmini                             Q-12                                  Bandra                              Bombay                                Maharastra                                400050              0

6 rows selected.
```

Problem statement:

Create the following tables using the schema given below and insert given data set accordingly.

Table Name- product_master

Description- Used to store product information

Column No	Column Name	Data Type	Size	Attributes
1	Product_no	Varchar2	6	Primary key, First letter must start with 'P'
2	Description	Varchar2	40	Not null
3	Profit_percent	Number	4,2	Not null
4	Unit_measure	Varchar2	10	Not null
5	Qty_on_hand	Number	8	Not null
6	Reorder_level	Number	8	Not null
7	Sell_price	Number	8,2	Not null, cannot be 0
8	Cost_price	Number	8,2	Not null, cannot be 0

Data of product_master table

Col-1	Col-2	Col-3	Col-4	Col-5	Col-6	Col-7	Col-8
P00001	1.44 Floppies	5	Piece	100	20	525	500
P03453	Monitors	6	Piece	10	3	12000	11280
P06734	Mouse	5	Piece	20	5	1050	1000
P07865	1.22 Floppies	5	Piece	100	20	525	500
P07868	Keyboard	2	Piece	10	3	3150	3050
P07885	CD Drive	2.5	Piece	10	3	5250	5100
P07965	540 HDD	4	Piece	10	3	8400	8000
P07975	1.44 Drive	5	Piece	10	3	1050	900
P08865	1.22 Drive	5	Piece	2	3	1025	850

Source code and Output:

```
Oracle SQL*Plus
File Edit Search Options Help
SQL> CREATE TABLE product_master (
2   product_no VARCHAR2(6) PRIMARY KEY,
3   description VARCHAR2(40) NOT NULL,
4   profit_percent NUMBER(4,2) NOT NULL,
5   unit_measure VARCHAR2(10) NOT NULL,
6   qty_on_hand NUMBER(8) NOT NULL,
7   reorder_level NUMBER(8) NOT NULL,
8   sell_price NUMBER(8,2) NOT NULL CHECK (sell_price != 0),
9   cost_price NUMBER(8,2) NOT NULL CHECK (cost_price != 0)
10 );

Table created.

SQL> desc product_master
Name
-----
PRODUCT_NO
DESCRIPTION
PROFIT_PERCENT
UNIT_MEASURE
QTY_ON_HAND
REORDER_LEVEL
SELL_PRICE
COST_PRICE
```

```

SQL> INSERT INTO product_master
  2 (product_no, description, profit_percent, unit_measure, qty_on_hand, reorder_level, sell_price,
  cost_price)
  3 VALUES
  4 ('P00001', '1.44 Floppies', 5, 'Piece', 100, 20, 525, 500);

1 row created.

SQL> INSERT INTO product_master
  2 (product_no, description, profit_percent, unit_measure, qty_on_hand, reorder_level, sell_price,
  cost_price)
  3 VALUES
  4 ('P03453', 'Monitors', 6, 'Piece', 10, 3, 12000, 11280);

1 row created.

SQL> INSERT INTO product_master
  2 (product_no, description, profit_percent, unit_measure, qty_on_hand, reorder_level, sell_price,
  cost_price)
  3 VALUES
  4 ('P06734', 'Mouse', 5, 'Piece', 20, 5, 1050, 1000);

1 row created.

SQL> INSERT INTO product_master
  2 (product_no, description, profit_percent, unit_measure, qty_on_hand, reorder_level, sell_price,
  cost_price)
  3 VALUES
  4 ('P07865', '1.22 Floppies', 5, 'Piece', 100, 20, 525, 500);

1 row created.

SQL> INSERT INTO product_master
  2 (product_no, description, profit_percent, unit_measure, qty_on_hand, reorder_level, sell_price,
  cost_price)
  3 VALUES
  4 ('P07868', 'Keyboard', 2, 'Piece', 10, 3, 3150, 3050);

1 row created.

SQL> INSERT INTO product_master
  2 (product_no, description, profit_percent, unit_measure, qty_on_hand, reorder_level, sell_price,
  cost_price)
  3 VALUES
  4 ('P07885', 'CD Drive', 2.5, 'Piece', 10, 3, 5250, 5100);

1 row created.

SQL> INSERT INTO product_master
  2 (product_no, description, profit_percent, unit_measure, qty_on_hand, reorder_level, sell_price,
  cost_price)
  3 VALUES
  4 ('P07965', '540 HDD', 4, 'Piece', 10, 3, 8400, 8000);

1 row created.

SQL> INSERT INTO product_master
  2 (product_no, description, profit_percent, unit_measure, qty_on_hand, reorder_level, sell_price,
  cost_price)
  3 VALUES
  4 ('P07975', '1.44 Drive', 5, 'Piece', 10, 3, 1050, 900);

1 row created.

SQL> INSERT INTO product_master
  2 (product_no, description, profit_percent, unit_measure, qty_on_hand, reorder_level, sell_price,
  cost_price)
  3 VALUES
  4 ('P08865', '1.22 Drive', 5, 'Piece', 2, 3, 1025, 850);

1 row created.

SQL> select * from product_master;

```

PRODUC	DESCRIPTION	PROFIT_PERCENT	UNIT_MEASU	QTY_ON_HAND	REORDER_LEVEL	SELL_PRICE	COST_PRICE
P00001	1.44 Floppies	5	Piece	100	20	525	500
P03453	Monitors	6	Piece	10	3	12000	11280
P06734	Mouse	5	Piece	20	5	1050	1000
P07865	1.22 Floppies	5	Piece	100	20	525	500
P07868	Keyboard	2	Piece	10	3	3150	3050
P07885	CD Drive	2.5	Piece	10	3	5250	5100
P07965	540 HDD	4	Piece	10	3	8400	8000
P07975	1.44 Drive	5	Piece	10	3	1050	900
P08865	1.22 Drive	5	Piece	2	3	1025	850

```

9 rows selected.

```

Problem statement:

Create the following tables using the schema given below and insert given data set accordingly.

Table Name- salesman_master:**Description- Used to store salesman working for company**

Column No	Column Name	Data Type	Size	Attributes
1	Salesman_no	Varchar2	6	Primary key, first letter must start with 'S'
2	Salesman_name	Varchar2	30	Not null
3	Address1	Varchar2	30	Not null
4	Address2	Varchar2	30	
5	City	Varchar2	20	
6	Pincode	Number	8	
7	State	Varchar2	20	
8	Sal_amt	Number	8, 2	Not null, cannot be 0

Data of salesman_master table

Col-1	Col-2	Col-3	Col-4	Col-5	Col-6	Col-7	Col-8
S001	Kiran	A/14	Worli	Bombay	400002	Maharastra	3000
S002	Manish	65	Nariman	Bombay	400001	Maharastra	3000
S003	Ravi	P-7	Bandra	Bombay	400032	Maharastra	3000
S004	Asish	A/5	Juhu	Bombay	400044	Maharastra	3000

Source code and Output:

```
Oracle SQL*Plus
File Edit Search Options Help
SQL> CREATE TABLE salesman_master (
2   salesman_no VARCHAR2(6) PRIMARY KEY,
3   salesman_name VARCHAR2(30) NOT NULL,
4   address1 VARCHAR2(30) NOT NULL,
5   address2 VARCHAR2(30),
6   city VARCHAR2(20),
7   pincode NUMBER(8),
8   state VARCHAR2(20),
9   sal_amt NUMBER(8,2) NOT NULL CHECK (sal_amt != 0)
10 );
Table created.
```

```

SQL> desc salesman_master
Name
-----
SALESMAN_NO
SALESMAN_NAME
ADDRESS1
ADDRESS2
CITY
PINCODE
STATE
SAL_AMT

SQL> INSERT INTO salesman_master
2 (salesman_no, salesman_name, address1, address2, city, pincode, state, sal_amt)
3 VALUES
4 ('S001', 'Kiran', 'A/14', 'Worli', 'Bombay', 400002, 'Maharashtra', 3000);

1 row created.

SQL> INSERT INTO salesman_master
2 (salesman_no, salesman_name, address1, address2, city, pincode, state, sal_amt)
3 VALUES
4 ('S002', 'Manish', '65', 'Nariman', 'Bombay', 400001, 'Maharashtra', 3000);

1 row created.

SQL> INSERT INTO salesman_master
2 (salesman_no, salesman_name, address1, address2, city, pincode, state, sal_amt)
3 VALUES
4 ('S003', 'Ravi', 'P-7', 'Bandra', 'Bombay', 400032, 'Maharashtra', 3000);

1 row created.

SQL> INSERT INTO salesman_master
2 (salesman_no, salesman_name, address1, address2, city, pincode, state, sal_amt)
3 VALUES
4 ('S004', 'Asish', 'A/5', 'Juhu', 'Bombay', 400044, 'Maharashtra', 3000);

1 row created.

SQL> select * from salesman_master;

SALESMAN_NO SALESMAN_NAME ADDRESS1 ADDRESS2 CITY PINCODE STATE SAL_AMT
-----
S001 Kiran A/14 Worli Bombay 400002 Maharashtra 3000
S002 Manish 65 Nariman Bombay 400001 Maharashtra 3000
S003 Ravi P-7 Bandra Bombay 400032 Maharashtra 3000
S004 Asish A/5 Juhu Bombay 400044 Maharashtra 3000

```

Problem statement:

Create the following tables using the schema given below and insert given data set accordingly.

Table Name- sales_order:

Description- Used to store client's orders

Column No	Column Name	Data Type	Size	Attributes
1	Order_no	Varchar2	6	Primary key, first letter must start with 'O'
2	Order_date	Date		
3	Client_no	Varchar2	6	Foreign key references Client_master table
4	Salesman_no	Varchar2	6	Foreign key references salesman_master table
5	Delivery_type	Char	1	Delivery part(P),full(F),Default 'F'
6	Bill_y_n	Char	1	
7	Delivery_date	Date		Cannot be less than Order_date
8	Order_status	Varchar2	10	Values('InProgress', 'Fullfilled', 'BackOrder', 'Cancelled')

Data of sales_order table

Col-1	Col-2	Col-3	Col-4	Col-5	Col- 6	Col-7	Col-8
O19001	12-Jan-96	C001	S001	F	N	20-Jan-96	InProcess
O19002	25-Jan-96	C002	S002	P	N	27-Jan-96	BackOrder
O46865	18-Feb-96	C003	S003	F	Y	20-Feb-96	Fullfilled
O19003	03-Apr-96	C001	S001	F	Y	07-Apr-96	Fullfilled
O46866	20-May-96	C004	S002	P	N	22-May-96	Cancelled
O19008	24-May-96	C005	S004	F	N	26-May-96	InProcess

Source code and Output:

```

Oracle SQL*Plus
File Edit Search Options Help
SQL> CREATE TABLE sales_order (
  2   order_no      VARCHAR2(6)    CONSTRAINT pk_sales_order PRIMARY KEY,
  3   order_date    DATE,
  4   client_no     VARCHAR2(6),
  5   salesman_no   VARCHAR2(6),
  6   delivery_type CHAR(1)        DEFAULT 'F',
  7   bill_y_n      CHAR(1),
  8   delivery_date DATE,
  9   order_status  VARCHAR2(10),
 10   CONSTRAINT ck_order_no CHECK (SUBSTR(order_no, 1, 1) = 'O'),
 11   CONSTRAINT ck_delivery_type CHECK (delivery_type IN ('P', 'F')),
 12   CONSTRAINT ck_delivery_date CHECK (delivery_date >= order_date),
 13   CONSTRAINT ck_order_status CHECK (order_status IN ('InProcess', 'Fullfilled', 'BackOrder',
'Cancelled'))
 14 );

Table created.

SQL> desc sales_order
Name
-----
ORDER_NO
ORDER_DATE
CLIENT_NO
SALESMAN_NO
DELIVERY_TYPE
BILL_Y_N
DELIVERY_DATE
ORDER_STATUS

SQL> INSERT INTO sales_order
  2 (order_no, order_date, client_no, salesman_no, delivery_type, bill_y_n, delivery_date, order_st
atus)
  3 VALUES
  4 ('O19001', TO_DATE('12-Jan-96', 'DD-Mon-YY'), 'C001', 'S001', 'F', 'N', TO_DATE('20-Jan-96', 'D
D-Mon-YY'), 'InProcess');

1 row created.

SQL> INSERT INTO sales_order
  2 (order_no, order_date, client_no, salesman_no, delivery_type, bill_y_n, delivery_date, order_st
atus)
  3 VALUES
  4 ('O19002', TO_DATE('25-Jan-96', 'DD-Mon-YY'), 'C002', 'S002', 'P', 'N', TO_DATE('27-Jan-96', 'D
D-Mon-YY'), 'BackOrder');

1 row created.

SQL> INSERT INTO sales_order
  2 (order_no, order_date, client_no, salesman_no, delivery_type, bill_y_n, delivery_date, order_st
atus)
  3 VALUES
  4 ('O46865', TO_DATE('18-Feb-96', 'DD-Mon-YY'), 'C003', 'S003', 'F', 'Y', TO_DATE('20-Feb-96', 'D
D-Mon-YY'), 'Fullfilled');

1 row created.

```

```

SQL> INSERT INTO sales_order
2 (order_no, order_date, client_no, salesman_no, delivery_type, bill_y_n, delivery_date, order_status)
3 VALUES
4 ('019003', TO_DATE('03-Apr-96', 'DD-Mon-YY'), 'C001', 'S001', 'F', 'Y', TO_DATE('07-Apr-96', 'DD-Mon-YY'), 'Fullfilled');

1 row created.

SQL> INSERT INTO sales_order
2 (order_no, order_date, client_no, salesman_no, delivery_type, bill_y_n, delivery_date, order_status)
3 VALUES
4 ('046866', TO_DATE('20-May-96', 'DD-Mon-YY'), 'C004', 'S002', 'P', 'N', TO_DATE('22-May-96', 'DD-Mon-YY'), 'Cancelled');

1 row created.

SQL> INSERT INTO sales_order
2 (order_no, order_date, client_no, salesman_no, delivery_type, bill_y_n, delivery_date, order_status)
3 VALUES
4 ('019008', TO_DATE('24-May-96', 'DD-Mon-YY'), 'C005', 'S004', 'F', 'N', TO_DATE('26-May-96', 'DD-Mon-YY'), 'InProgress');

1 row created.

SQL> select * from sales_order;

ORDER_ ORDER_DAT CLIENT SALESM D B DELIVERY_ ORDER_STAT
-----
019001 12-JAN-96 C001 S001 F N 20-JAN-96 InProcess
019002 25-JAN-96 C002 S002 P N 27-JAN-96 BackOrder
046865 18-FEB-96 C003 S003 F Y 20-FEB-96 Fullfilled
019003 03-APR-96 C001 S001 F Y 07-APR-96 Fullfilled
046866 20-MAY-96 C004 S002 P N 22-MAY-96 Cancelled
019008 24-MAY-96 C005 S004 F N 26-MAY-96 InProcess

6 rows selected.

```

Problem statement:

Create the following tables using the schema given below and insert given data set accordingly.

Table Name- sales_order_details:

Description- Used to store client's orders with details of each product ordered

Colmn No	Column Name	Data Type	Size	Attributes
1	Order_no	Varchar2	6	Foreign key referencessales_order table
2	Product_no	Varchar2	6	Foreign key references product_master table
3	Qty_ordered	Number	8	
4	Qty_disp	Number	8	
5	Product_rate	Number	10, 2	

Data of sales_order_details

Col-1	Col-2	Col-3	Col-4	Col-5
O19001	P00001	4	4	525
O19001	P07965	2	1	8400
O19001	P07885	2	1	5250
O19002	P00001	10	0	525
O46865	P07868	3	3	3150
O46865	P07885	3	1	5250
O46865	P00001	10	10	525
O46865	P03453	4	4	1050
O19003	P03453	2	2	1050
O19003	P06734	1	1	12000
O46866	P07965	1	0	8400
O46866	P07975	1	0	1050
O19008	P00001	10	5	525
O19008	P07975	5	3	1050

Source code and Output:

```

Oracle SQL*Plus
File Edit Search Options Help

SQL> CREATE TABLE sales_order_details (
  2   order_no VARCHAR2(6),
  3   product_no VARCHAR2(6),
  4   qty_ordered NUMBER(8),
  5   qty_disp NUMBER(8),
  6   product_rate NUMBER(10,2),
  7   PRIMARY KEY (order_no, product_no),
  8   FOREIGN KEY (order_no) REFERENCES sales_order (order_no),
  9   FOREIGN KEY (product_no) REFERENCES product_master (product_no)
 10 );

Table created.

SQL> desc sales_order_details
Name
-----
ORDER_NO
PRODUCT_NO
QTY_ORDERED
QTY_DISP
PRODUCT_RATE

SQL> INSERT INTO sales_order_details
  2 (order_no, product_no, qty_ordered, qty_disp, product_rate)
  3 VALUES
  4 ('O19001', 'P00001', 4, 4, 525);

1 row created.

SQL> INSERT INTO sales_order_details
  2 (order_no, product_no, qty_ordered, qty_disp, product_rate)
  3 VALUES
  4 ('O19001', 'P07965', 2, 1, 8400);

1 row created.

SQL> INSERT INTO sales_order_details
  2 (order_no, product_no, qty_ordered, qty_disp, product_rate)
  3 VALUES
  4 ('O19001', 'P07885', 2, 1, 5250);

1 row created.

SQL> INSERT INTO sales_order_details
  2 (order_no, product_no, qty_ordered, qty_disp, product_rate)
  3 VALUES
  4 ('O19002', 'P00001', 10, 0, 525);

1 row created.

SQL> INSERT INTO sales_order_details
  2 (order_no, product_no, qty_ordered, qty_disp, product_rate)
  3 VALUES
  4 ('O46865', 'P07868', 3, 3, 3150);

1 row created.

SQL> INSERT INTO sales_order_details
  2 (order_no, product_no, qty_ordered, qty_disp, product_rate)
  3 VALUES
  4 ('O46865', 'P07885', 3, 1, 5250);

1 row created.

```

```

SQL> INSERT INTO sales_order_details
2 (order_no, product_no, qty_ordered, qty_disp, product_rate)
3 VALUES
4 ('046865', 'P00001', 10, 10, 525);

1 row created.

SQL> INSERT INTO sales_order_details
2 (order_no, product_no, qty_ordered, qty_disp, product_rate)
3 VALUES
4 ('046865', 'P03453', 4, 4, 1050);

1 row created.

SQL> INSERT INTO sales_order_details
2 (order_no, product_no, qty_ordered, qty_disp, product_rate)
3 VALUES
4 ('019003', 'P03453', 2, 2, 1050);

1 row created.

SQL> INSERT INTO sales_order_details
2 (order_no, product_no, qty_ordered, qty_disp, product_rate)
3 VALUES
4 ('019003', 'P06734', 1, 1, 12000);

1 row created.

SQL> INSERT INTO sales_order_details
2 (order_no, product_no, qty_ordered, qty_disp, product_rate)
3 VALUES
4 ('046866', 'P07965', 1, 0, 8400);

1 row created.

SQL> INSERT INTO sales_order_details
2 (order_no, product_no, qty_ordered, qty_disp, product_rate)
3 VALUES
4 ('046866', 'P07975', 1, 0, 1050);

1 row created.

SQL> INSERT INTO sales_order_details
2 (order_no, product_no, qty_ordered, qty_disp, product_rate)
3 VALUES
4 ('019008', 'P00001', 10, 5, 525);

1 row created.

SQL> INSERT INTO sales_order_details
2 (order_no, product_no, qty_ordered, qty_disp, product_rate)
3 VALUES
4 ('019008', 'P07975', 5, 3, 1050);

1 row created.

SQL> select * from sales_order_details;

ORDER_  PRODUCE  QTY_ORDERED  QTY_DISP  PRODUCT_RATE
-----
019001  P00001      4           4          525
019001  P07965      2           1          8400
019001  P07885      2           1          5250
019002  P00001     10           0          525
046865  P07868      3           3          3150
046865  P07885      3           1          5250
046865  P00001     10          10          525
046865  P03453      4           4          1050
019003  P03453      2           2          1050
019003  P06734      1           1         12000
046866  P07965      1           0          8400
046866  P07975      1           0          1050
019008  P00001     10           5          525
019008  P07975      5           3          1050

14 rows selected.

```

Assignment No: 02

Title: Data Manipulation (DML)-I

Problem 1: Find the names of all clients having 'A' as the second letter in their names.

Source Code:

```
SQL> select name from client_master where name like '_a%';
```

Output:

```
NAME
-----
VandanaSatiwal
BasuNavindgi
Ravi Sreedharan

SQL> |
```

Problem 2: Find out the clients who do not stay in a city whose first letter is 'b'.

Source Code:

```
SQL> select name from client_master where city not like 'b%';
```

Output:

```
NAME
-----
Ivan Bayross
VandanaSatiwal
PramadaJaguste
BasuNavindgi
Ravi Sreedharan
Rukmini

6 rows selected.
```

Problem 3 : List the names and city of all clients who have exactly 12 characters in length and starts with 'I'.

Source Code:

```
SQL> select name, city from client_master where length(name)=12 and name like 'i%';
```

Output:

```
no rows selected
```

Problem 4: Find the list of all clients who stay in 'Bombay' or 'Delhi'.

Source Code:

```
SQL> SELECT * FROM client_master WHERE city IN ('Bombay', 'Delhi');
```

Output:

CLIENT NAME		ADDRESS1		
ADDRESS2		CITY	STATE	PINCODE
BALANCE_DUE				
C001	Ivan Bayross Worli 15000	Bombay	P-76 Maharashtra	400054
C004	BasuNavindgi Nariman 0	Bombay	A/12 Maharashtra	400056
CLIENT NAME		ADDRESS1		
ADDRESS2		CITY	STATE	PINCODE
BALANCE_DUE				
C005	Ravi Sreedharan Rajnagar 2000	Delhi	B/34 Dehli	100001
C006	Rukmini Bandra	Bombay	Q-12 Maharashtra	400050
CLIENT NAME		ADDRESS1		
ADDRESS2		CITY	STATE	PINCODE
BALANCE_DUE				

Problem 5: Print the list of all clients whose balance_due is greater than value 10,000.

Source Code:

```
SQL> select name from client_master where BALANCE_Due>10000;
```

Output:

```
NAME
-----
Ivan Bayross
```

Problem 6: Print the information from sales_order table for orders places in the month of January.

Source Code:

```
SQL> select * from sales_order where to_char(order_date,'mm')='01';
```

Output:

```
ORDER_  ORDER_DAT  CLIENT  SALESM  D  B  DELIVERY_  ORDER_STAT
-----  -
019001  12-JAN-96  C001    S001    F  N  20-JAN-96  InProcess
019002  25-JAN-96  C002    S002    P  N  27-JAN-96  BackOrder
SQL> |
```

Problem 7: Display the order information for client_no 'C001' and 'C002'

Source Code:

```
SQL> SELECT * FROM SALES_ORDER WHERE CLIENT_NO in('C001', 'C002');
```

Output:

```
ORDER_  ORDER_DAT  CLIENT  SALESM  D  B  DELIVERY_  ORDER_STAT
-----  -
019001  12-JAN-96  C001    S001    F  N  20-JAN-96  InProcess
019002  25-JAN-96  C002    S002    P  N  27-JAN-96  BackOrder
019003  03-APR-96  C001    S001    F  Y  07-APR-96  Fullfiled
```

Problem 8: Find products whose selling price greater than 2000 and less than 5000.

Source Code:

```
SQL> SELECT * FROM PRODUCT_MASTER WHERE SELL_PRICE>2000 AND SELL_PRICE<5000;
```

Output:

```
PRODUC  DESCRIPTION                                PROFIT_PERCENT  UNIT_MEASU
-----  -
QTY_ON_HAND REORDER_LEVEL  SELL_PRICE  COST_PRICE
-----  -
P07868  Keyboard
      10              3        3150        3050              2 Piece
```

Problem 9: Find products whose selling price is more than 1500. Calculate a new selling price as original selling price*1.15. Rename the new column in the above query is New_price.

Source Code:

```
SQL> SELECT sell_price,sell_price*1.15 as NEW_PRICE from product_master where Sell_price>1500.00;
```

Output:

SELL_PRICE	NEW_PRICE
12000	13800
3150	3622.5
5250	6037.5
8400	9660

Problem 10: List the names, city and state of clients who are not in the state of 'Maharastra'.

Source Code:

```
SQL> SELECT NAME, CITY, STATE FROM CLIENT_MASTER WHERE STATE!='Maharastra';
```

Output:

NAME	CITY	STATE
VandanaSatiwal	Madras	TamilNadu
PramadaJaguste	Kolkata	West Bengal
Ravi Sreedharan	Delhi	Dehli

Problem 11: Display the month(in alphabets) and date when the order must be delivered.

Source Code:

```
SQL> SELECT TO_CHAR(DELIVERY_DATE, 'month-dd') FROM SALES_ORDER;
```

Output:

```
TO_CHAR(DELIVERY_DATE, 'MONTH-DD')
-----
january -20
january -27
february -20
april -07
may -22
may -26

6 rows selected.
```

Problem 12: Display the Order_date in the format 'DD-Month-YY' e.g 12-February-13.

Source Code:

```
SQL> SELECT TO_CHAR(ORDER_DATE, 'DD-Month-YY') O_DATE FROM SALES_ORDER;
```

Output:

```
O_DATE
-----
12-January -96
25-January -96
18-February -96
03-April    -96
20-May      -96
24-May      -96

6 rows selected.
```

Problem 13: Find the date, 15 days after today's date

Source Code:

```
SQL> SELECT SYSDATE+15 NEW_DATE FROM DUAL;
```

Output:

```
NEW_DATE
-----
11-MAR-25
```

Assignment No: 03

Title: Data Manipulation (DML)-II

Problem 1: Count the total number of orders.

Source Code:

```
SQL> SELECT COUNT(*) FROM SALES_ORDER;
```

Output:

```
COUNT(*)
-----
        6
```

Problem 2: Calculate the average sale price of all the products.

Source Code:

```
SQL> SELECT AVG(Cost_Price) from product_master;
```

Output:

```
AVG(COST_PRICE)
-----
    3464.44444
```

Problem 3: Count the number of products having price greater than or equal to 1500.

Source Code:

```
SQL> SELECT COUNT(PRODUCT_NO) FROM PRODUCT_MASTER WHERE SELL_PRICE>=1500;
```

Output:

```
COUNT(PRODUCT_NO)
-----
                4
```


Problem 4: Determine the maximum and minimum product prices. Rename the output as max_price and min_price respectively.

Source Code:

```
SQL> SELECT MAX(SELL_PRICE) MAX_PRICE, MIN(SELL_PRICE) MIN_PRICE FROM PRODUCT_MASTER;
```

Output:

```
MAX_PRICE  MIN_PRICE
-----
12000      525
```

Problem 5: Create the following tables and insert the data as mentioned below:

a) Emp table

Column	Null	Type
EMPNO	NOT NULL	NUMBER(4,0)
ENAME		VARCHAR2(10)
JOB		VARCHAR2(9)
MGR		NUMBER(4,0)
HIREDATE	-	DATE
SAL	-	NUMBER(7,2)
COMM	-	NUMBER(7,2)
DEPTNO	NOT NULL	NUMBER(2,0)

Data in Emp table [Value absent means NULL]

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10

b) Dept table:

Column	Null	Type
deptno	NOT NULL	NUMBER(2,0)
dname	NOT NULL	VARCHAR2(14)
loc	NOT NULL	VARCHAR2(13)



Laboratory Assignments

Sessi
MCKVIE/CS

Data in Dept table

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON



Laboratory Assignments

Data in Dept table

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

c) Salgrade table

Column	Null	Type
grade		NUMBER,
losal		NUMBER,
hisal		NUMBER

Data in Salgrade table:

GRADE	LOSAL	HISAL
1	700	1200
2	1201	1400
3	1401	2000
4	2001	3000
5	3001	9999

EMP TABLE:

Source Code:

```
SQL> CREATE TABLE Emp (  
2 EMPNO NUMBER(4,0) NOT NULL,  
3 ENAME VARCHAR2(10),  
4 JOB VARCHAR2(9),  
5 MGR NUMBER(4,0),  
6 HIREDATE DATE,  
7 SAL NUMBER(7,2),  
8 COMM NUMBER(7,2),  
9 DEPTNO NUMBER(2,0) NOT NULL,  
10 PRIMARY KEY (EMPNO)  
11 );
```

Output:

Table created.

Source Code:

```
SQL> INSERT INTO Emp (EMPNO, ENAME, JOB, MGR, HIREDATE, SAL, COMM, DEPTNO) VALUES  
2 (7369, 'SMITH', 'CLERK', 7902, TO_DATE('17-DEC-80', 'DD-MON-YY'), 800, NULL, 20),  
3 (7499, 'ALLEN', 'SALESMAN', 7698, TO_DATE('20-FEB-81', 'DD-MON-YY'), 1600, 300, 30),  
4 (7521, 'WARD', 'SALESMAN', 7698, TO_DATE('22-FEB-81', 'DD-MON-YY'), 1250, 500, 30),  
5 (7566, 'JONES', 'MANAGER', 7839, TO_DATE('02-APR-81', 'DD-MON-YY'), 2975, NULL, 20),  
6 (7654, 'MARTIN', 'SALESMAN', 7698, TO_DATE('28-SEP-81', 'DD-MON-YY'), 1250, 1400, 30),  
7 (7698, 'BLAKE', 'MANAGER', 7839, TO_DATE('01-MAY-81', 'DD-MON-YY'), 2850, NULL, 30),  
8 (7782, 'CLARK', 'MANAGER', 7839, TO_DATE('09-JUN-81', 'DD-MON-YY'), 2450, NULL, 10),  
9 (7788, 'SCOTT', 'ANALYST', 7566, TO_DATE('19-APR-87', 'DD-MON-YY'), 3000, NULL, 20),  
10 (7839, 'KING', 'PRESIDENT', NULL, TO_DATE('17-NOV-81', 'DD-MON-YY'), 5000, NULL, 10),  
11 (7844, 'TURNER', 'SALESMAN', 7698, TO_DATE('08-SEP-81', 'DD-MON-YY'), 1500, 0, 30),  
12 (7876, 'ADAMS', 'CLERK', 7788, TO_DATE('23-MAY-87', 'DD-MON-YY'), 1100, NULL, 20),  
13 (7900, 'JAMES', 'CLERK', 7698, TO_DATE('03-DEC-81', 'DD-MON-YY'), 950, NULL, 30),  
14 (7902, 'FORD', 'ANALYST', 7566, TO_DATE('03-DEC-81', 'DD-MON-YY'), 3000, NULL, 20),  
15 (7934, 'MILLER', 'CLERK', 7782, TO_DATE('23-JAN-82', 'DD-MON-YY'), 1300, NULL, 10);
```

Output:

```
SQL> SELECT * from emp ;
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM
DEPTNO						
7369	SMITH	CLERK	7902	17-DEC-80	800	
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500
DEPTNO						
7566	JONES	MANAGER	7839	02-APR-81	2975	
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400
7698	BLAKE	MANAGER	7839	01-MAY-81	2850	
DEPTNO						
7782	CLARK	MANAGER	7839	09-JUN-81	2450	
7788	SCOTT	ANALYST	7566	19-APR-87	3000	
7839	KING	PRESIDENT		17-NOV-81	5000	
DEPTNO						
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0
7876	ADAMS	CLERK	7788	23-MAY-87	1100	

DEPT TABLE:

Source Code:

```
SQL> CREATE TABLE Dept (  
2     DEPTNO NUMBER(2,0) NOT NULL,  
3     DNAME VARCHAR2(14) NOT NULL,  
4     LOC VARCHAR2(13) NOT NULL,  
5     PRIMARY KEY (DEPTNO)  
6 );
```

Output:

```
Table created.
```

Source Code:

```
SQL> INSERT INTO Dept (DEPTNO, DNAME, LOC) VALUES  
2 (10, 'ACCOUNTING', 'NEW YORK'),  
3 (20, 'RESEARCH', 'DALLAS'),  
4 (30, 'SALES', 'CHICAGO'),  
5 (40, 'OPERATIONS', 'BOSTON');
```

Output:

```
SQL> select * from dept;
```

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

SALGRADE TABLE:

Source Code:

```
SQL> CREATE TABLE Salgrade (  
2     GRADE NUMBER,  
3     LOSAL NUMBER,  
4     HISAL NUMBER  
5 );
```

Output:

```
Table created.
```

Source Code:

```
SQL> INSERT INTO Salgrade (GRADE, LOSAL, HISAL) VALUES  
2 (1, 700, 1200),  
3 (2, 1201, 1400),  
4 (3, 1401, 2000),  
5 (4, 2001, 3000),  
6 (5, 3001, 9999);
```

Output:

```
SQL> select * from salgrade;
```

GRADE	LOSAL	HISAL
1	700	1200
2	1201	1400
3	1401	2000
4	2001	3000
5	3001	9999

Problem 6:a) List all department no, employee no, manager no. from emp table.

Source Code:

```
SQL> SELECT DEPTNO, EMPNO, MGR FROM Emp;
```

Output:

DEPTNO	EMPNO	MGR
20	7369	7902
30	7499	7698
30	7521	7698
20	7566	7839
30	7654	7698
30	7698	7839
10	7782	7839
20	7788	7566
10	7839	
30	7844	7698
20	7876	7788
DEPTNO	EMPNO	MGR
30	7900	7698
20	7902	7566
10	7934	7782

14 rows selected.

b) List all department name and location from dept table.

Source Code:

```
SQL> select dname,loc from dept;
```

Output:

DNAME	LOC
ACCOUNTING	NEW YORK
RESEARCH	DALLAS
SALES	CHICAGO
OPERATIONS	BOSTON

c) List the employees belong to the department 20.

Source Code:

```
SQL> select ename from emp where deptno=20;
```

Output:

ENAME
SMITH
JONES
SCOTT
ADAMS
FORD

d) List the name and salary of the employee whose salary is more than 2500.

Source Code:

```
SQL> select ename,sal from emp where sal>2500;
```

Output:

ENAME	SAL
JONES	2975
BLAKE	2850
SCOTT	3000
KING	5000
FORD	3000

e) List the details of the employee who have joined before end of July 81.

Source Code:

```
SQL> select * from emp where hiredate<'31-JULY-81';
```

Output:

```
no rows selected
```

f) List the name of the employees who are not manager.

Source Code:

```
SQL> select ename from emp where job!='Manager';
```

Output:

```
ENAME
-----
SMITH
ALLEN
WARD
JONES
MARTIN
BLAKE
CLARK
SCOTT
KING
TURNER
ADAMS

ENAME
-----
JAMES
FORD
MILLER

14 rows selected.
```

g) List the name of employees whose name end with 'S'.

Source Code:

```
SQL> select ename from emp where ename like '%S';
```

Output:

```
ENAME
-----
JONES
ADAMS
JAMES
```

h) List the name of employees whose name has exactly 5 characters long.

Source Code:

```
SQL> select ename from emp where length(ename)=5;
```

Output:

```
ENAME
-----
SMITH
ALLEN
JONES
BLAKE
CLARK
SCOTT
ADAMS
JAMES

8 rows selected.
```

i) List the name of employees whose name having 'R' as the third character.

Source Code:

```
SQL> select ename from emp where ename like '__R%';
```

Output:

```
ENAME
-----
WARD
MARTIN
TURNER
FORD
```

j) List all employee names and their salaries, whose salary lies between 1500/- and 3500/- both inclusive.

Source Code:

```
SQL> select ename, sal from emp where sal between 1500 and 3500;
```

Output:

ENAME	SAL
ALLEN	1600
JONES	2975
BLAKE	2850
CLARK	2450
SCOTT	3000
TURNER	1500
FORD	3000

7 rows selected.

k) List all employee names and their and their manager whose manager is 7902 or 7566 Or 7789.

Source Code:

```
SQL> select ename, mgr from emp where mgr=7902 or mgr=7566 or mgr=7789;
```

Output:

ENAME	MGR
SMITH	7902
SCOTT	7566
FORD	7566

l) List all employees which start with either J or T.

Source Code:

```
SQL> select ename from emp where ename like 'J%' or ename like 'T%';
```

Output:

ENAME
JONES
TURNER
JAMES

m) List all employee names and jobs, whose job title includes M or P.

Source Code:

```
SQL> select ename, job from emp where job like '%M%' or ename like '%P%';
```

Output:

ENAME	JOB
ALLEN	SALESMAN
WARD	SALESMAN
JONES	MANAGER
MARTIN	SALESMAN
BLAKE	MANAGER
CLARK	MANAGER
TURNER	SALESMAN

7 rows selected.

n) List all jobs available in employee table.

Source Code:

```
SQL> select distinct job from emp;
```

Output:

JOB
CLERK
SALESMAN
MANAGER
ANALYST
PRESIDENT

o) List all employees who belong to the department 10 or 20.

Source Code:

```
SQL> select ename from emp where deptno=10 or deptno=20;
```

Output:

ENAME
SMITH
JONES
CLARK
SCOTT
KING
ADAMS
FORD
MILLER

p) List all employee names, salary and 15% raise in salary.

Source Code:

```
SQL> select ename , sal,1.15*sal raised_sal from emp;
```

Output:

ENAME	SAL	RAISED_SAL
SMITH	800	920
ALLEN	1600	1840
WARD	1250	1437.5
JONES	2975	3421.25
MARTIN	1250	1437.5
BLAKE	2850	3277.5
CLARK	2450	2817.5
SCOTT	3000	3450
KING	5000	5750
TURNER	1500	1725
ADAMS	1100	1265
ENAME	SAL	RAISED_SAL
JAMES	950	1092.5
FORD	3000	3450
MILLER	1300	1495

q) List minimum, maximum, average salaries of employee.

Source Code:

```
SQL> select min(sal) minimum_sal, max(sal) maximum_sal ,avg(sal) average_sal from emp;
```

Output:

MINIMUM_SAL	MAXIMUM_SAL	AVERAGE_SAL
800	5000	2073.21429

r) Find how many job titles are available in employee table.

Source Code:

```
SQL> SELECT COUNT(DISTINCT JOB) AS Job_Count FROM Emp;
```

Output:

JOB_COUNT
5

s) What is the difference between maximum and minimum salaries of employees in the organization?

Source Code:

```
SQL> select max(sal)-min(sal) as salary_difference from emp;
```

Output:

```
SALARY_DIFFERENCE
-----
                4200
```

t) Find how much amount the company is spending towards salaries.

Source Code:

```
SQL> SELECT SUM(SAL) AS Total_Salary_Expense FROM Emp;
```

Output:

```
TOTAL_SALARY_EXPENSE
-----
                29025
```

u) Display name of employees with deptno. 20.

Source Code:

```
SQL> select ename from emp where deptno=20;
```

Output:

```
ENAME
-----
SMITH
JONES
SCOTT
ADAMS
FORD
```

v) List ename who are not eligible for commission.

Source Code:

```
SQL> select ename from emp where comm is null or comm=0;
```

Output:

```
ENAME
-----
SMITH
JONES
BLAKE
CLARK
SCOTT
KING
TURNER
ADAMS
JAMES
FORD
MILLER
```

w) Find no.of dept in employee table.

Source Code:

```
SQL> SELECT COUNT(DISTINCT DEPTNO) AS Total_Departments FROM Emp;
```

Output:

TOTAL_DEPARTMENTS
3

x) List ename and designation of the employee who does not report to anybody.

Source Code:

```
SQL> SELECT ENAME, JOB FROM Emp WHERE MGR IS NULL;
```

Output:

ENAME	JOB
KING	PRESIDENT

y) List the name of the employees who are less than 38 years old in the organization with respect to today's date.

Source Code:

```
SQL> select ename, hiredate from emp where(sysdate-hiredate)/365<38;
```

Output:

ENAME	HIREDATE
SMITH	17-DEC-80
ALLEN	20-FEB-81
WARD	22-FEB-81
JONES	02-APR-81
MARTIN	28-SEP-81
BLAKE	01-MAY-81
CLARK	09-JUN-81
SCOTT	19-APR-87
KING	17-NOV-81
TURNER	08-SEP-81
ADAMS	23-MAY-87

ENAME	HIREDATE
JAMES	03-DEC-81
FORD	03-DEC-81
MILLER	23-JAN-82

Assignment-4

Data Manipulation (DML)-II I: Use of GROUP BY, HAVING, ORDER BY and different in-built functions:

1.Display empno, ename, sal in ascending order of salary from emp table.

```
SQL> select empno,ename,sal from EMP order by sal ASC;
```

EMPNO	ENAME	SAL
7369	SMITH	800
7900	JAMES	950
7876	ADAMS	1100
7654	MARTIN	1250
7521	WARD	1250
7934	MILLER	1300
7844	TURNER	1500
7499	ALLEN	1600
7782	CLARK	2450
7698	BLAKE	2850
7566	JONES	2975
7902	FORD	3000
7788	SCOTT	3000
7839	KING	5000

14 rows selected.

2.List ename, sal, job and deptno in descending order of deptno and salary.

```
SQL> select ename,sal,job,deptno from EMP order by deptno DESC,sal DESC;
```

ENAME	SAL	JOB	DEPTNO
BLAKE	2850	MANAGER	30
ALLEN	1600	SALESMAN	30
TURNER	1500	SALESMAN	30
WARD	1250	SALESMAN	30
MARTIN	1250	SALESMAN	30
JAMES	950	CLERK	30
FORD	3000	ANALYST	20
SCOTT	3000	ANALYST	20
JONES	2975	MANAGER	20
ADAMS	1100	CLERK	20
SMITH	800	CLERK	20
KING	5000	PRESIDENT	10
CLARK	2450	MANAGER	10
MILLER	1300	CLERK	10

14 rows selected.

3. List ename, sal, PF, HRA, DA and GROSS in ascending order of Gross. [Here PF is 12% of sal, HRA is 15% of sal, DA is 90% of sal and GROSS is sum of sal, PF, HRA, DA].

```
SQL> select ename, sal,
2 sal*0.12 as PF,
3 sal*0.15 as HRA,
4 sal*0.90 as DA,
5 sal+(sal*0.12)+(sal*0.15)+(sal*0.90) as GROSS from emp order by GROSS ASC;
```

ENAME	SAL	PF	HRA	DA	GROSS
SMITH	800	96	120	720	1736
JAMES	950	114	142.5	855	2061.5
ADAMS	1100	132	165	990	2387
MARTIN	1250	150	187.5	1125	2712.5
WARD	1250	150	187.5	1125	2712.5
MILLER	1300	156	195	1170	2821
TURNER	1500	180	225	1350	3255
ALLEN	1600	192	240	1440	3472
CLARK	2450	294	367.5	2205	5316.5
BLAKE	2850	342	427.5	2565	6184.5
JONES	2975	357	446.25	2677.5	6455.75

ENAME	SAL	PF	HRA	DA	GROSS
FORD	3000	360	450	2700	6510
SCOTT	3000	360	450	2700	6510
KING	5000	600	750	4500	10850

14 rows selected.

4. List the maximum salary of employee working as a salesman

```
SQL> select max(sal) from emp where JOB='SALESMAN';
```

MAX(SAL)
1600

5. List the average salary and no of employees working in dept 20.

```
SQL> select avg(sal), COUNT(*) from emp where deptno=20;
```

AVG(SAL)	COUNT(*)
2175	5

6.Display deptno, no. of employees in each department.

```
SQL> select deptno,COUNT(*) from emp group by deptno;
```

DEPTNO	COUNT(*)
30	6
20	5
10	3

7.List deptno, total salary payable in each department.

```
SQL> select deptno,sum(sal) from emp group by deptno;
```

DEPTNO	SUM(SAL)
30	9400
20	10875
10	8750

8.List jobs and the no of employees in each job in descending order of no. of employees.

```
SQL> select job,COUNT(*) from emp group by job order by count(*) desc;
```

JOB	COUNT(*)
CLERK	4
SALESMAN	4
MANAGER	3
ANALYST	2
PRESIDENT	1

9.List total, maximum, minimum and average salary of employee's job wise.

```
SQL> select job,sum(sal),max(sal),min(sal),avg(sal) from emp group by job;
```

JOB	SUM(SAL)	MAX(SAL)	MIN(SAL)	AVG(SAL)
CLERK	4150	1300	800	1037.5
SALESMAN	5600	1600	1250	1400
PRESIDENT	5000	5000	5000	5000
MANAGER	8275	2975	2450	2758.33333
ANALYST	6000	3000	3000	3000

10. List the average salary for each job excluding manager.

```
SQL> select job, avg(sal) from emp where job != 'MANAGER' group by job;
```

JOB	AVG(SAL)
CLERK	1037.5
SALESMAN	1400
PRESIDENT	5000
ANALYST	3000

11. List total, maximum, minimum and average salary of employee's job-wise for dept no. 20 only.

```
SQL> select job, max(sal), min(sal), avg(sal) from emp where deptno=20 group by job;
```

JOB	MAX(SAL)	MIN(SAL)	AVG(SAL)
CLERK	1100	800	950
MANAGER	2975	2975	2975
ANALYST	3000	3000	3000

12. List average monthly salary for each job within department.

```
SQL> select deptno, job, AVG(sal) from emp group by deptno, job;
```

DEPTNO	JOB	AVG(SAL)
20	MANAGER	2975
20	CLERK	950
30	SALESMAN	1400
30	CLERK	950
10	PRESIDENT	5000
30	MANAGER	2850
10	CLERK	1300
10	MANAGER	2450
20	ANALYST	3000

9 rows selected.

13. List average salary for all departments where more than 5 people are working.

```
SQL> select deptno, AVG(sal) from emp group by deptno having count(*) > 5;
```

DEPTNO	AVG(SAL)
30	1566.66667

14. List jobs of all employees where maximum salary is greater than or equal to 5000.

```
SQL> select distinct job from emp where job in (select job from emp group by job having max(sal) >=5000);
```

JOB
PRESIDENT

15. Display total, maximum, minimum and average salaries of employee's job-wise for department 20 and list only those rows having average salary greater than 1000.

```
SQL> select job,sum(sal),max(sal),min(sal),avg(sal) from emp where deptno =20 group by job having avg(sal) >1000;
```

JOB	SUM(SAL)	MAX(SAL)	MIN(SAL)	AVG(SAL)
MANAGER	2975	2975	2975	2975
ANALYST	6000	3000	3000	3000

16. Display total, maximum, minimum and average salaries of employee's job-wise for department 20 and list only those rows having average salary greater than 1000 and arrange the above output in descending order of total salary.

```
SQL> select job,sum(sal),max(sal),min(sal),avg(sal) from emp where deptno =20 group by job having avg(sal) >1000 order by sum(sal) desc;
```

JOB	SUM(SAL)	MAX(SAL)	MIN(SAL)	AVG(SAL)
ANALYST	6000	3000	3000	3000
MANAGER	2975	2975	2975	2975

17. Calculate the average of the maximum salaries of all the departments from emp table.

```
SQL> select avg(max_sal) from (select max(sal) as max_sal from emp group by deptno);
```

AVG(MAX_SAL)
3616.66667

18. Display the standard deviation (sd) of salary for each job type having sd >0 from emp table.

```
SQL> select job,stddev(sal) from emp group by job having stddev(sal)>0;
```

JOB	STDDEV(SAL)
CLERK	213.600094
SALESMAN	177.951304
MANAGER	274.241378

19.Count no. of employees whose commission is greater than 300.

```
SQL> select count(*) from emp where comm>300;
```

COUNT(*)
2

20.Display sum of commission for each department after substituting 100 in commission if it is NULL and order the result in descending order of department.

```
SQL> select deptno,SUM(COALESCE(comm,100)) from emp group by deptno order by deptno desc;
```

DEPTNO	SUM(COALESCE(COMM,100))
30	2400
20	500
10	300

21.Display no. of manager present in employee table.

```
SQL> select count(*) from emp where job ='MANAGER';
```

COUNT(*)
3

22.List of employee names and commissions, substituting "Not Applicable" if the employee receives no commission for those whose name has contained a "M" and order this result as descending order of name.

```
SQL> SELECT ENAME, COALESCE(TO_CHAR(COMM), 'NOT APPLICABLE') AS COMMISSION FROM EMP WHERE ENAME LIKE '%M%' ORDER BY ENAME DESC;
```

ENAME	COMMISSION
SMITH	NOT APPLICABLE
MILLER	NOT APPLICABLE
MARTIN	1400
JAMES	NOT APPLICABLE
ADAMS	NOT APPLICABLE

23. List names, salary and commission of employees whose name has contained a “M” when the income of some employees is made up of salary plus commission, or just salary, depending on whether the comm column of employees is null or not and order the result as ascending order of name.

```
SQL> SELECT ENAME, SAL, COMM FROM EMP WHERE ENAME LIKE '%M%' ORDER BY ENAME ASC;
```

ENAME	SAL	COMM
ADAMS	1100	
JAMES	950	
MARTIN	1250	1400
MILLER	1300	
SMITH	800	

24. Display the name of the employee where first character of each name is capital one

```
SQL> SELECT ENAME FROM EMP WHERE SUBSTR(ENAME,1,1) = UPPER(SUBSTR(ENAME,1,1));
```

ENAME
KING
BLAKE
CLARK
JONES
SCOTT
FORD
SMITH
ALLEN
WARD
MARTIN
TURNER

ENAME
ADAMS
JAMES
MILLER

25. Select the substring of 3 characters long starting from 2nd character of job type from emp table when job is ‘SALESMAN’.

```
SQL> SELECT SUBSTR(JOB,2,3) FROM EMP WHERE JOB = 'SALESMAN';
```

SUBSTR(JOB,2
ALE
ALE
ALE
ALE

Assignment No: 05

Title: Join of multiple tables; Nested queries - Sub-queries

Problem 1: Display empno, emoname, dname and loc from emp and dept table.

Source Code:

```
SQL> SELECT e.empno, e.ename, d.dname, d.loc
2 FROM emp e
3 JOIN dept d ON e.deptno = d.deptno;
```

Output:

EMPNO	ENAME	DNAME	LOC
7369	SMITH	RESEARCH	DALLAS
7499	ALLEN	SALES	CHICAGO
7521	WARD	SALES	CHICAGO
7566	JONES	RESEARCH	DALLAS
7654	MARTIN	SALES	CHICAGO
7698	BLAKE	SALES	CHICAGO
7782	CLARK	ACCOUNTING	NEW YORK
7788	SCOTT	RESEARCH	DALLAS
7839	KING	ACCOUNTING	NEW YORK
7844	TURNER	SALES	CHICAGO
7876	ADAMS	RESEARCH	DALLAS
7900	JAMES	SALES	CHICAGO
7902	FORD	RESEARCH	DALLAS
7934	MILLER	ACCOUNTING	NEW YORK

Problem 2: Display empno, empname and their deptno, deptname and loc from emp and dept table.

Source Code:

```
SQL> SELECT e.empno, e.ename, e.deptno, d.dname, d.loc
2 FROM emp e
3 JOIN dept d ON e.deptno = d.deptno;
```

Output:

EMPNO	ENAME	DEPTNO	DNAME	LOC
7369	SMITH	20	RESEARCH	DALLAS
7499	ALLEN	30	SALES	CHICAGO
7521	WARD	30	SALES	CHICAGO
7566	JONES	20	RESEARCH	DALLAS
7654	MARTIN	30	SALES	CHICAGO
7698	BLAKE	30	SALES	CHICAGO
7782	CLARK	10	ACCOUNTING	NEW YORK
7788	SCOTT	20	RESEARCH	DALLAS
7839	KING	10	ACCOUNTING	NEW YORK
7844	TURNER	30	SALES	CHICAGO
7876	ADAMS	20	RESEARCH	DALLAS
7900	JAMES	30	SALES	CHICAGO
7902	FORD	20	RESEARCH	DALLAS
7934	MILLER	10	ACCOUNTING	NEW YORK

Problem 3: Display the list of employees working in each department and with this display the department information even if no employee belongs to that department.

Source Code:

```
SQL> SELECT d.dname, d.loc, e.ename
2 FROM dept d
3 LEFT JOIN emp e ON d.deptno = e.deptno;
```

Output:

DNAME	LOC	ENAME
RESEARCH	DALLAS	SMITH
SALES	CHICAGO	ALLEN
SALES	CHICAGO	WARD
RESEARCH	DALLAS	JONES
SALES	CHICAGO	MARTIN
SALES	CHICAGO	BLAKE
ACCOUNTING	NEW YORK	CLARK
RESEARCH	DALLAS	SCOTT
ACCOUNTING	NEW YORK	KING
SALES	CHICAGO	TURNER
RESEARCH	DALLAS	ADAMS
DNAME	LOC	ENAME
SALES	CHICAGO	JAMES
RESEARCH	DALLAS	FORD
ACCOUNTING	NEW YORK	MILLER
OPERATIONS	BOSTON	

Problem 4: List the employee name and their corresponding manager name from emp table.

Source Code:

```
SQL> SELECT e.ename AS Employee, m.ename AS Manager
2 FROM emp e
3 JOIN emp m ON e.mgr = m.empno;
```

Output:

EMPLOYEE	MANAGER
SCOTT	JONES
FORD	JONES
ALLEN	BLAKE
WARD	BLAKE
MARTIN	BLAKE
TURNER	BLAKE
JAMES	BLAKE
MILLER	CLARK
ADAMS	SCOTT
JONES	KING
BLAKE	KING
EMPLOYEE	MANAGER
CLARK	KING
SMITH	FORD

Problem 5: List the employee name and their corresponding manager name from emp table along this select that employee who have no manager.

Source Code:

```
SQL> SELECT e.ename AS Employee, m.ename AS Manager
2 FROM emp e
3 LEFT JOIN emp m ON e.mgr = m.empno;
```

Output:

EMPLOYEE	MANAGER
SCOTT	JONES
FORD	JONES
ALLEN	BLAKE
WARD	BLAKE
MARTIN	BLAKE
TURNER	BLAKE
JAMES	BLAKE
MILLER	CLARK
ADAMS	SCOTT
JONES	KING
BLAKE	KING
EMPLOYEE	MANAGER
CLARK	KING
SMITH	FORD
KING	

Problem 6: List all employees who joined the company before their manager.

Source Code:

```
SQL> SELECT e.ename AS Employee, m.ename AS Manager
2 FROM emp e
3 JOIN emp m ON e.mgr = m.empno
4 WHERE e.hiredate < m.hiredate;
```

Output:

EMPLOYEE	MANAGER
ALLEN	BLAKE
WARD	BLAKE
JONES	KING
BLAKE	KING
CLARK	KING
SMITH	FORD

Problem 7: Find names, job and salaries of all employees and also his boss.

Source Code:

```
SQL> SELECT e.ename AS Employee, e.job, e.sal, m.ename AS Manager
2 FROM emp e
3 JOIN emp m ON e.mgr = m.empno;
```

Output:

EMPLOYEE	JOB	SAL	MANAGER
SCOTT	ANALYST	3000	JONES
FORD	ANALYST	3000	JONES
ALLEN	SALESMAN	1600	BLAKE
WARD	SALESMAN	1250	BLAKE
MARTIN	SALESMAN	1250	BLAKE
TURNER	SALESMAN	1500	BLAKE
JAMES	CLERK	950	BLAKE
MILLER	CLERK	1300	CLARK
ADAMS	CLERK	1100	SCOTT
JONES	MANAGER	2975	KING
BLAKE	MANAGER	2850	KING
EMPLOYEE	JOB	SAL	MANAGER
CLARK	MANAGER	2450	KING
SMITH	CLERK	800	FORD

Problem 8: Find the names of those employees who earn more than their boss.

Source Code:

```
SQL> SELECT e.ename AS Employee
2   FROM emp e
3   JOIN emp m ON e.mgr = m.empno
4  WHERE e.sal > m.sal;
```

Output:

```
EMPLOYEE
-----
SCOTT
FORD
```

Problem 9: How much Miller needs to earn to be in BLAKE's Grade?.

Source Code:

```
SQL> SELECT (b.sal - m.sal) AS Amount_needed
2   FROM emp m
3   JOIN emp b ON b.ename = 'BLAKE'
4  WHERE m.ename = 'MILLER';
```

Output:

```
AMOUNT_NEEDED
-----
1550
```

Problem 10: List the employee name and their corresponding manager name from emp table where employee name includes an "A" and display will be appeared as in the example given below.

Employees and Their Managers
Allen works for Blake
Blake works for King

Source Code:

```
SQL> SELECT e.ename || ' works for ' || m.ename AS Employee_Manager
2 FROM emp e
3 JOIN emp m ON e.mgr = m.empno
4 WHERE e.ename LIKE '%A%';
```

Output:

```
EMPLOYEE_MANAGER
-----
ALLEN works for BLAKE
WARD works for BLAKE
MARTIN works for BLAKE
JAMES works for BLAKE
ADAMS works for SCOTT
BLAKE works for KING
CLARK works for KING
```

Problem 11: Display different designations present in department 20 and 30.

Source Code:

```
SQL> SELECT DISTINCT e.job
2 FROM emp e
3 WHERE e.deptno IN (20, 30);
```

Output:

```
JOB
-----
CLERK
SALESMAN
MANAGER
ANALYST
```

Problem 12: List the job common to department 20 and 30.

Source Code:

```
SQL> SELECT e.job
2 FROM emp e
3 WHERE e.deptno = 20
4 AND e.job IN (SELECT job FROM emp WHERE deptno = 30);
```

Output:

```
JOB
-----
MANAGER
CLERK
CLERK
```

Problem 13: List the jobs that are unique to department 20 compare to other departments present in emp table.

Source Code:

```
SQL> SELECT e.job
2 FROM emp e
3 WHERE e.deptno = 20
4 AND e.job NOT IN (SELECT job FROM emp WHERE deptno != 20);
```

Output:

```
JOB
-----
ANALYST
ANALYST
```

Problem 14: List the employees belong to department of MILLER.

Source Code:

```
SQL> SELECT e.ename
2 FROM emp e
3 WHERE e.deptno = (SELECT deptno FROM emp WHERE ename = 'MILLER');
```

Output:

```
ENAME
-----
CLARK
KING
MILLER
```

Problem 15: Display all employee details whose salary is greater than average salary of employees whose date of joining is before 1st April 81.

Source Code:

```
SQL> SELECT * FROM emp e
2 WHERE e.sal > (SELECT AVG(sal) FROM emp WHERE hiredate < TO_DATE('1981-04-01', 'YYYY-MM-DD'));
```

Output:

```
no rows selected
```

Problem 16: List the job with highest average salary.

Source Code:

```
SQL> SELECT job
  2 FROM emp
  3 GROUP BY job
  4 HAVING AVG(sal) = (SELECT MAX(avg_sal) FROM (SELECT AVG(sal) AS avg_sal FROM emp GROUP BY job));
```

Output:

```
JOB
-----
PRESIDENT
```

Problem 17: Find the details of the employees of the department whose manager no is 7698.

Source Code:

```
SQL> SELECT *
  2 FROM emp
  3 WHERE deptno = (SELECT deptno FROM emp WHERE empno = 7698);
```

Output:

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM

DEPTNO						

7499 30	ALLEN	SALESMAN	7698	20-FEB-81	1600	300
7521 30	WARD	SALESMAN	7698	22-FEB-81	1250	500
7654 30	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM

DEPTNO						

7698	BLAKE	MANAGER	7839	01-MAY-81	2850	

Problem 18: List the names of the employees who earn lowest salary in each department.

Source Code:

```
SQL> SELECT E.ENAME, E.deptno, E.Sal
  2 FROM emp E
  3 WHERE Sal = (
  4     SELECT MIN(Sal)
  5     FROM emp
  6     WHERE deptno = E.deptno
  7 );
```

Output:

ENAME	DEPTNO	SAL
SMITH	20	800
JAMES	30	950
MILLER	10	1300

Problem 19: List the employee details whose salary greater than the average salary for their department.

Source Code:

```
SQL> SELECT *  
2 FROM emp e  
3 WHERE e.sal > (SELECT AVG(sal) FROM emp WHERE deptno = e.deptno);
```

Output:

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM
DEPTNO						
7499 30	ALLEN	SALESMAN	7698	20-FEB-81	1600	300
7566 20	JONES	MANAGER	7839	02-APR-81	2975	
7698 30	BLAKE	MANAGER	7839	01-MAY-81	2850	
EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM
DEPTNO						
7788 20	SCOTT	ANALYST	7566	19-APR-87	3000	
7839	KING	PRESIDENT		17-NOV-81	5000	

Problem 20: List all employees who work in DALLAS and earn more than any employee working in CHICAGO.

Source Code:

```
SQL> SELECT e.ename  
2 FROM emp e  
3 WHERE e.deptno = (SELECT deptno FROM dept WHERE loc = 'DALLAS')  
4 AND e.sal > (SELECT MAX(sal) FROM emp WHERE deptno = (SELECT deptno FROM dept WHERE loc = 'CHICAGO'));
```

Output:

```
ENAME  
-----  
JONES  
SCOTT  
FORD
```

Assignment No: 06

Title: Database Modification

Problem 1: Create two table emp_new and dept_new as the same structure and data as of emp and dept table..

Source Code:

```
SQL> CREATE TABLE emp_new AS  
2  SELECT * FROM emp;
```

```
SQL> CREATE TABLE dept_new AS  
2  SELECT * FROM dept;
```

Output:

```
Table created.
```

```
Table created.
```

Problem 2: Modify dept_new table so that deptno will be the PRIMARY key.

Source Code:

```
SQL> ALTER TABLE dept_new  
2  ADD CONSTRAINT pk_deptno PRIMARY KEY (deptno);
```

Output:

```
Table altered.
```

Problem 3: Add a column named, 'Grade' (which holds grade of the employees) with CHAR(1) to the existing emp_new table.

Source code:

```
SQL> ALTER TABLE emp_new  
2  ADD Grade CHAR(1);
```

Output:

```
Table altered.
```

Problem 4: Modify emp_new table to add a constraint 'emp_noPri' as PRIMARY key on empno attribute.

Source code:

```
SQL> ALTER TABLE emp_new  
2 ADD CONSTRAINT emp_noPri PRIMARY KEY (empno);
```

Output:

```
Table altered.
```

Problem 5: Add constraints REFERENCES to deptno of emp_new table referring deptno of dept_new table.

Source code:

```
SQL> ALTER TABLE emp_new  
2 ADD CONSTRAINT fk_deptno FOREIGN KEY (deptno)  
3 REFERENCES dept_new(deptno);
```

Output:

```
Table altered.
```

```
SQL> desc emp_new;  
Name                               Null?    Type  
-----  
EMPNO                               NOT NULL NUMBER(4)  
ENAME                               VARCHAR2(10)  
JOB                                 VARCHAR2(9)  
MGR                                 NUMBER(4)  
HIREDATE                           DATE  
SAL                                 NUMBER(7, 2)  
COMM                               NUMBER(7, 2)  
DEPTNO                             NOT NULL NUMBER(2)  
GRADE                              CHAR(1)
```

Problem 6: Modify the emp_new table so that we can't store the salary of the employee which is less than 2500.

Source code:

```
SQL> ALTER TABLE emp_new  
2 ADD CONSTRAINT chk_sal CHECK (sal >= 2500) NOVALIDATE;
```

Output:

```
Table altered.
```

Problem 7: Modify the sal column of emp_new table to NOT NULL and increase its size to 10.

Source code:

```
SQL> ALTER TABLE EMP_NEW MODIFY SAL NULL;
Table altered.

SQL> ALTER TABLE emp_new ADD sal_temp NUMBER(10);
Table altered.

SQL> UPDATE emp_new SET sal_temp = sal;
14 rows updated.

SQL> ALTER TABLE emp_new DROP COLUMN sal;
Table altered.

SQL> ALTER TABLE emp_new RENAME COLUMN sal_temp TO sal;
Table altered.

SQL> ALTER TABLE emp_new MODIFY sal NUMBER(10) NOT NULL;
Table altered.
```

Output:

```
Table altered.
```

Problem 8: Modify the ename column of emp_new table to increase its width to VARCHAR(35).

Source code:

```
SQL> ALTER TABLE emp_new MODIFY ename VARCHAR2(35);
```

Output:

```
Table altered.
```

```
SQL> desc emp_new;
Name                               Null?    Type
-----
EMPNO                               NOT NULL NUMBER(4)
ENAME                               VARCHAR2(35)
JOB                                 VARCHAR2(9)
MGR                                 NUMBER(4)
HIREDATE                             DATE
SAL                                 NUMBER(7,2)
COMM                                NUMBER(7,2)
DEPTNO                             NOT NULL NUMBER(2)
GRADE                               CHAR(1)
```

Problem 9: Drop the PRIMARY key constraint from emp_new table.

Source code:

```
SQL> ALTER TABLE emp_new DROP PRIMARY KEY;
```

Output:

```
Table altered.
```

Problem 10: Drop the primary key constraint of dept_new table so that it will also dropped the foreign key constraint on emp_new_deptno.

Source code:

```
SQL> ALTER TABLE dept_new DROP PRIMARY KEY CASCADE;
```

Output:

```
Table altered.
```

Problem 11: Display all column name and constraint name of emp_new table.

Source code:

```
SQL> SELECT cols.column_name, cons.constraint_name
2  FROM user_constraints cons
3  JOIN user_cons_columns cols ON cons.constraint_name = cols.constraint_name
4  WHERE cons.table_name = 'EMP_NEW';
```

Output:

```
COLUMN_NAME
-----
CONSTRAINT_NAME
-----
EMPNO
SYS_C008512

DEPTNO
SYS_C008513
```

Problem 12: Disable the primary key constraint of dept_new table.

Source code:

```
SQL> ALTER TABLE dept_new
2  DISABLE CONSTRAINT dept_new_pk;
```

Output:

```
Table altered.
```

Problem 13: Create table emp_new1 and dept_new1 from emp and dept respectively (also consider that there is a referential integrity between the tables using deptno attribute).

Source code:

```
SQL> CREATE TABLE dept_new1 AS SELECT * FROM dept;
```



```
SQL>  
SQL> CREATE TABLE emp_new1 AS SELECT * FROM emp;
```

Output:

```
Table created.
```

```
Table created.
```

Problem 14: Now try to drop the dept_new1 table and give a comment on your output.

Source code:

```
SQL> DROP TABLE dept_new1;
```

Output:

```
Table dropped.
```

Problem 15: Add the comment 'Employee Information of XYZ Company' on emp table.

Source code:

```
SQL> COMMENT ON TABLE emp IS 'Employee Information of XYZ Company';
```

Output:

```
Comment created.
```

Problem 16: Add the comment 'Unique depart of XYZ Company' to deptno column of dept table.

Source code:

```
SQL> COMMENT ON COLUMN dept.deptno IS 'Unique depart of XYZ Company';
```

Output:

```
Comment created.
```

Problem 17: How to show the comments of a particular column of a table?

Source code:

```
SQL> SELECT * FROM user_col_comments WHERE table_name = 'DEPT' AND column_name = 'DEPTNO';
```

Output:

```
TABLE_NAME
-----
COLUMN_NAME
-----
COMMENTS
-----
ORIGIN_CON_ID
-----
DEPT
DEPTNO
Unique depart of XYZ Company
1
```

Problem 18: How to show the comments on a particular a table?

Source code:

```
SQL> SELECT * FROM user_tab_comments WHERE table_name = 'EMP';
```

Output:

```
TABLE_NAME
-----
TABLE_TYPE
-----
COMMENTS
-----
ORIGIN_CON_ID
-----
EMP
TABLE
Employee Information of XYZ Company
1
```

Problem 19: Remove the comment on emp table.

Source code:

```
SQL> COMMENT ON TABLE emp IS '';
```

Output:

```
Comment created.
```

Problem 20: Change the name of the emp table as emp_change table.

Source code:

```
SQL> RENAME emp TO emp_change;
```

```
SQL> desc emp_change;
```

Output:

```
Table renamed.
```

```
SQL> desc emp_change;
Name                                     Null?    Type
-----
EMPNO                                   NOT NULL NUMBER(4)
ENAME                                  VARCHAR2(10)
JOB                                    VARCHAR2(9)
MGR                                    NUMBER(4)
HIREDATE                               DATE
SAL                                    NUMBER(7,2)
COMM                                   NUMBER(7,2)
DEPTNO                                 NOT NULL NUMBER(2)
```

Problem 21: Drop the emp_new1 table so that the table structure will remain there in the database table.

Source code:

```
SQL> TRUNCATE TABLE emp_new1;
```

Output:

```
Table truncated.
```

Problem 22: Drop the dept_new1 table permanently so the table structure will no longer present in the database.

Source code:

```
SQL> DROP TABLE dept_new1 ;
```

Output:

```
Table dropped.
```

Problem 23 : Drop the Gade column from emp_new table..

Source code:

```
• ALTER TABLE emp_new DROP COLUMN Grade;
```

Output:

```
Table altered.
```

Problem 24 : Make the comm. column of emp_new table as unused

Source code:

```
SQL> ALTER TABLE emp_new SET UNUSED (comm);
```

Output:

```
Table altered.
```

Problem 25: How can we display the marked unused columns of a emp_new table.

Source code:

```
SQL> SELECT table_name, column_name  
2 FROM user_unused_col_tabs  
3 WHERE table_name = 'EMP_NEW';  
SELECT table_name, column_name
```

Problem 26: Drop the unused columns of emp_new table.

Source code:

```
SQL> ALTER TABLE emp_new DROP UNUSED COLUMNS;
```

Output:

```
Table altered.
```

Problem 27: Rename the comm. column of emp_new table to commission.

Source code:

```
SQL> ALTER TABLE emp_new RENAME COLUMN sal TO commission;
```

Output:

```
Table altered.
```

Assignment No: 07

Title: VIEWS

Problem 1: Create a view D_10EMP containing details of employees in department 10.

Source Code:

```
SQL> CREATE VIEW D_10EMP AS
2  SELECT *
3  FROM emp
4  WHERE deptno = 10;
```

Output:

```
View created.
```

Problem 2: Create a view Dept_Summary containing Dname, MINSal, MAXSal, AVGSal.

Source Code:

```
SQL> CREATE VIEW Dept_Summary AS
2  SELECT d.dname,
3         MIN(e.sal) AS MINSal,
4         MAX(e.sal) AS MAXSal,
5         AVG(e.sal) AS AVGSAL
6  FROM emp e
7  JOIN dept d ON e.deptno = d.deptno
8  GROUP BY d.dname;
```

Output:

```
View created.
```

Problem 3: Create a view Emp_Data which would validate entry of any INSERT/UPDATE command so that Sal figure would be between 10000 to 20000 and MGR would have to be a valid Empno, and Deptno must exists on Dept table.

Source Code:

```
SQL> CREATE VIEW Emp_Data AS
2  SELECT *
3  FROM emp
4  WHERE sal BETWEEN 10000 AND 20000;
```

Output:

```
View created.
```

Problem 4: Create a view which restricts data access to a valid employee's own details on the Emp table and access time to office hours (09-17 and MON- SAT).

Source Code:

```
SQL> CREATE VIEW Emp_SelfView AS
2  SELECT *
3  FROM emp
4  WHERE empno = USER -- Oracle specific
5  AND TO_CHAR(SYSDATE, 'DY') IN ('MON', 'TUE', 'WED', 'THU', 'FRI', 'SAT')
6  AND TO_CHAR(SYSDATE, 'HH24') BETWEEN 9 AND 17;
```

Output:

```
View created.
```

Problem 5: Create a view EMPLOYEES from emp table which contains empno, ename, job, dept and validate the deptno. that must be in Dept table.

Source Code:

```
SQL> CREATE VIEW EMPLOYEES AS
2  SELECT e.empno, e.ename, e.job, e.deptno
3  FROM emp e
4  WHERE EXISTS (
5      SELECT 1
6      FROM dept d
7      WHERE d.deptno = e.deptno
8  );
```

Output:

```
View created.
```

Problem 6: Create a READ only view EMP_READ from emp table which contains job, minsal, maxsal.

Source Code:

```
SQL> CREATE VIEW EMP_READ AS
2  SELECT job, MIN(sal) AS minsal, MAX(sal) AS maxsal
3  FROM emp
4  GROUP BY job
5  WITH READ ONLY;
```

Output:

```
View created.
```

Problem 7:

7. Table Client_master

Column Name	Data type	Size	Attributes
Client_no	Varchar2	8	Primary Key
Name	Varchar2	20	Not Null
Address1	Varchar2	20	Not Null
Address2	Varchar2	20	
City	Varchar2	15	
Pincode	Varchar2	8	
Bal_due	Number	8,3	

A. Create a view vw_client_master using Client_no, Name, Address1 and Bal_due
i. Insert at least 3 records to vw_client_master.
ii. Update a record to vw_client_master.
iii. Delete a record from vw_client_master.
And check that the above operation if the base table is affected or not.

B. Create a view Vw_sales_det using Order_no, Order_date, Product_no, Qty_ordered, and order_status for all order which have already marked as 'Backorder'. (Using the tables sales_order, sales_order_details).
i. Insert a record to vw_sales_det.
ii. Update the client_no for a particular order_no.
iii. Delete a record.
iv. Remove the views from database.

A. Create a view vw_client_master using Client_no, Name, Address1 and Bal_due

Source Code:

```
SQL> CREATE VIEW vw_client_master AS
2  SELECT Client_no, Name, Address1, Balance_due
3  FROM Client_master;
```

Output:

```
View created.
```

i. Insert at least 3 records to vw_client_master.

Source Code:

```
SQL> INSERT INTO Client_master VALUES ('C0012', 'Alice', 'Addr1', 'Addr2', 'CityA', 'West Bengal', '560001', 1500.00);
1 row created.
SQL> INSERT INTO Client_master VALUES ('C0013', 'Bob', 'AddrX', 'AddrY', 'CityB', 'Bihar', '560002', 1200.00);
1 row created.
SQL> INSERT INTO Client_master VALUES ('C0014', 'Charlie', 'Addr3', 'Addr4', 'CityC', 'UP', '560003', 800.00);
1 row created.
```

Output:

```
1 row created.
```

ii. Update a record to vw_client_master.

Source Code:

```
SQL> UPDATE vw_client_master
2  SET Balance_due = 2000.00
3  WHERE Client_no = 'C001';
```

Output:

```
1 row updated.
```

CLIENT NAME		ADDRESS1		
ADDRESS2		CITY	STATE	PINCODE
BALANCE_DUE				
C001	Ivan Bayross	P-76		
Worli		Bombay	Maharastra	400054
	2000			

iii. Delete a record from vw_client_master.

Source Code:

```
SQL> DELETE FROM vw_client_master
2 WHERE Client_no = 'C002';
```

Output:

```
1 row deleted.
```

```
SQL > |
```

B. Source Code:

```
SQL> CREATE VIEW vw_sales AS
2 SELECT s.order_no, s.order_date, d.product_no, d.qty_ordered, s.order_status
3 FROM sales_order s
4 JOIN sales_order_details d ON s.order_no = d.order_no
5 WHERE s.order_status = 'Backorder';
```

Output:

```
View created.
```

```
SQL> |
```

B.i) Source Code:

```
SQL>
SQL> INSERT INTO sales_order_details VALUES ('019001', 'P00001', 4, 4, 525);
```

```
SQL> INSERT INTO SALES_ORDER VALUES('019001', '12-JAN-96', 'C001', 'S001', 'F', 'N', '20-JAN-96', 'InProgress');
INSERT INTO SALES_ORDER VALUES('019001', '12-JAN-96', 'C001', 'S001', 'F', 'N', '20-JAN-96', 'InProgress')
```

Output:

```
1 row created.
```

B.ii) Source Code:

```
SQL> UPDATE sales_order
2 SET client_no = 'C003'
3 WHERE order_no = '01001';
```


Output:

```
rows updated.
```

B.iii) Source Code:

```
SQL> DELETE FROM sales_order_details  
2 WHERE order_no = '019003' AND product_no = 'P03453';
```

Output:

```
1 row deleted.
```

B.iv) Source Code:

```
SQL> DROP VIEW vw_sales;
```

Output:

```
View dropped.
```